

QuadPath: A Four-Expert Branch Architecture for Surgical Organ Segmentation

Ashton Davis & Ramez Mosad

May 2026

1 Abstract

Segmentation of laparoscopic surgical images presents unique challenges due to tool occlusions, bleeding, deformable organ surfaces, specular reflections, and visual similarity between anatomical structures. Prior research by Guo et al. introduced a three-expert branch model for laparoscopic image segmentation that kept feature encodings mostly independent until the final fusion stage. This design allowed each convolutional expert branch to focus on different types of visual information, such as texture, boundary shape, and curved anatomical structures. Although the original model showed promising results, some organ classes still had low segmentation accuracy, with mIoU scores as low as 21% for difficult structures. These limitations suggested that the model could benefit from an additional expert branch capable of capturing broader contextual information. In this project, we extend the original TriPath architecture [2] by adding a MiT-B0 Vision Transformer branch, creating a QuadPath model for multi-organ surgical image segmentation. The goal of this added branch is to improve segmentation mask quality by combining local convolutional features with transformer-based global feature representations. We found that adding the MiT-B0 branch improved overall segmentation performance compared to the original TriPathEB-WF baseline. The QuadPath model achieved higher average mIoU, mDice, and NSD scores, while also reducing MASD, indicating improvements in both region overlap and boundary accuracy. Per-organ results showed especially strong gains for difficult structures such as the ureter, vesicular glands, inferior mesenteric artery, and pancreas. These findings suggest that transformer-based global context can complement existing expert branches and improve laparoscopic organ segmentation, particularly for smaller or visually challenging anatomical structures.

2 Introduction

Laparoscopic surgery has become an increasingly adopted practice within medicine due to a variety of benefits stemming from the minimally invasive nature of these surgeries compared to traditional methods [1]. While this new method offers benefits like reductions in recovery times and shorter hospital stays it also introduces new complications that differ from the expectations learned during traditional surgery. Key concerns

such as limited visibility and reduced feedback compared to traditional surgical methods create issues in the adoption of this process. While training for this has become increasingly common, training often requires intensive supervision by experienced operators who are currently in short supply and use a screen that offers minimal feedback or guidance in the training process [1].

In exploring new routes for creating better feedback for applications in training and live surgery, image and video segmentation is being explored to understand how to best create a channel for clearer feedback and guidance. Segmentation however faces critical challenges such rapid camera motions, deformable organ surfaces, surgical smoke, specular reflections, tool occlusions, bleeding, and overall extreme complexity within the images [2]. These visual conditions create severe variability in appearance and fundamentally challenges automated segmentation models. Unlike natural domains where lighting is more likely to be controlled and cleared of some of these obstacles, laparoscopic images present a constantly unsure, shifting environment where the same organ may look vastly different across frames. Due to this the model must rely on more geometrical patterns to adhere to thin curved boundaries rather than relying heavily on learning the colors and textures, but there is no singular feature extraction model that currently is truly suited for the diverse geometric patterns present throughout laparoscopic images [2]. All singular feature extraction models tend to lean towards capturing a key structural pattern but typical fused model architectures collapse the features into a singular pathway earlier within the model reducing the ability for each model to truly develop the rich representation of the key structural pattern it leans towards [2].

Building on prior work, this study extends the TriEB model, by extending to a fourth expert branch aiming to add a global context expert through the addition of the MiT-B0 vision transformer. While TriEB demonstrated promising improvements over a baseline UNet, it underperformed on geometrically complex thin structures such as the ureter, inferior mesenteric artery, and vesicular glands. This suggested that the cnn based expert branches lack sufficient global context for segmentation of geometrically challenging organs. The proposed QuadEB model addresses this limitation through the integration of a pre-trained ViT branch. Evaluated on the DSAD laparoscopic dataset, QuadEB Achieved a mean mIoU of 0.569 across all 11 organs compared to 0.510 for TriEb. The substantial gains were observed in the key underperforming organs

such as ureter (+0.1496 mIoU), vesicular glands (+0.0712) and the inferior mesenteric artery (+0.0444).

3 Related Work

Multi-branch architectures have been widely explored to enhance representational diversity in computer vision. While other designs expand feature diversity, features from different models are typically merged early, which creates tangled representations that limit the individual contribution of each module [2]. To combat this, Guo et al. introduced the idea of Expert Branches in their TriEB model in which singular feature extraction models run simultaneously on the same data and do not interact until the ending fusion point [2]. TriEB demonstrated that independently training a CNN, deformable convolution (DCN), and dynamic snake convolution (DSC) within the UNet backbone with no interaction of these streams until the ending fusion point improved segmentation performance on the DSAD dataset. Critically this study constrained the models to just untrained convolutional modules and they emphasized parameter efficiency.

Vision transformers have emerged as a dominant model for dense complex prediction tasks [3]. Segformer introduced the Mix Transformer encoder, including MiT-B0. This produces hierarchical multi-scale features without positional encoding and achieves strong performance under ImageNet pretraining [4]. However, as noted in prior surgical imaging literature, surgical datasets are often small and are highly-domain specific leading to difficulty with ViT models learning low-level geometric features making it weaker at segmentation on deformable and complex boundaries [2].

The DSAD dataset provides pixel-wise segmentation annotations for eleven abdominal anatomical structures. It provides 13,195 laparoscopic images sourced from 32 procedures and serves as the primary dataset used throughout this model [5].

4 Problem Definition and Solution Design

Binary segmentation of laparoscopic surgical images presents key issues such as rapid camera motion, highly deformable organ surfaces, specular reflections, smoke, bleeding, tool occlusions, and thin, sharply curved anatomical structures that interfere with accuracy of segmentation masks. While the TriEB models included an expert aimed at local context and texture (CNN), a nonrigid deformation and spatial misalignment expert (DCN) and an expert on contours (DSN), it was clear there is key information missing. The original TriEB model deliberately was constrained to convolutional experts without pre-training. This left global contextual as a missing component that limited the efficacy of the segmentation masks.

This work addresses that gap by introducing QuadEb, a four-branch extension of TriEB that incorporates a pretrained MiT-B0 vision transformer as a fourth expert branch. The goal is to determine whether adding a global context expert

within the Expert Branch framework improves segmentation quality across the DSAD dataset and particularly across complex structures in which TriEB underperformed.

5 Methodology

5.1 Data Collection and Preprocessing

This project used the DSAD laparoscopic surgical image dataset, which contains surgical images paired with annotated organ segmentation masks. The dataset includes multiple organs, such as the abdominal wall, colon, inferior mesenteric artery, intestinal veins, liver, pancreas, small intestine, spleen, stomach, ureter, and vesicular glands. These images were used to train and evaluate the model on single-organ binary segmentation tasks, where each target organ was segmented against the background.

Before training, the image-mask pairs were organized by organ class so that each experiment focused on one anatomical structure at a time. The raw images were not modified directly; instead, processed copies were created for model training. Each image was resized to match the input size required by the model, and the corresponding mask was resized and matched to the correct target organ. Image inputs were also normalized using ImageNet mean and standard deviation values to make them compatible with the pretrained MiT-B0 branch. This preprocessing ensured that the CNN, DCN, DSC, MiT-B0 branches could receive consistent input data during training.

5.2 QuadPath Architecture Overview

This project extends the original TriPath expert branch architecture into a four-branch model, referred to as QuadPath. The original TriPath model used three separate expert branches to extract different types of features from laparoscopic surgical images: a standard CNN branch for regional texture, a deformable convolution branch for nonrigid and misaligned structures, and a dynamic snake convolution branch for curved anatomical patterns. Our contribution was to add a fourth expert branch based on a pretrained Vision Transformer backbone, specifically MiT-B0, to introduce global contextual feature extraction into the existing architecture.

The motivation for this design was that laparoscopic surgical images are visually complex. Organs can have similar textures and colors, and structures may be partially hidden by tools, blood, lighting artifacts, or specular reflections. While convolutional branches are effective for learning local and shape-based features, they may struggle when an organ’s identity depends on broader scene context. The added MiT-B0 branch was intended to complement the existing three branches by helping the model understand relationships across larger regions of the image.

In the final QuadPath architecture, all four branches process the same input image. Each branch produces multi-scale feature maps at different depths. These features are then combined with a shared UNet-style decoder. The decoder recon-

structs the final segmentation mask by using both the deepest fused features at the bottleneck and skip connection features from earlier encoder levels. Refer to figure 1 for a full visual overview of the architecture.

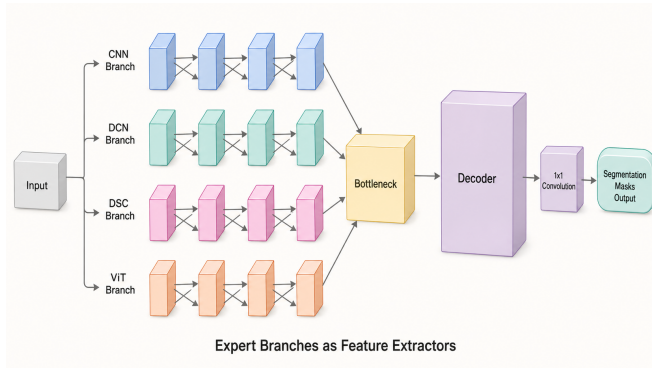


Figure 1: Visual representation of the QuadPath architecture showing the key models and their fusion point.

5.3 Implementation of the Vision Transformer Branch

The fourth expert branch was implemented using MiT-B0, a lightweight backbone from the SegFormer family of models. MiT-B0 was selected because it is a hierarchical transformer encoder that produces multi-scale feature maps, making it more compatible with encoder-decoder segmentation architectures than a standard flat Vision Transformer. A traditional ViT often produces a single sequence of patch embeddings, but MiT-B0 outputs features at multiple spatial resolutions. This made it a better fit for our UNet-style architecture, which relies on features from multiple encoder stages.

The MiT-B0 branch was loaded using the Hugging Face ‘transformers’ library. Since the model was pretrained on ImageNet, the input images were normalized using ImageNet mean and standard deviation. This preprocessing step was important because the pretrained weights expect images to follow a similar input distribution to the data used during pretraining.

In our implementation, the MiT-B0 branch acted as an encoder-only component. We did not use the original SegFormer decode head because our goal was not to use SegFormer as a standalone segmentation model inside the architecture. Instead, we used MiT-B0 only as a feature extractor. Its extracted feature maps were passed into our QuadPath decoder alongside the feature maps from CNN, DCN, and DSC branches.

A key reason for using MiT-B0 was its ability to capture global context. In simple terms, global context means the model can use the information from broader areas of the image rather than only nearby pixels. This is useful in surgical images because a small or visually unclear structure may be easier to identify when the model also considers its surrounding anatomy and location in the full surgical scene.

5.4 Feature Alignment and Skip Connections

One of the most challenging aspects of implementation was integrating the MiT-B0 features with the existing TriPath architecture. The original TriPath branches were convolution-based and produced feature maps with channel dimensions and spatial resolutions expected by the UNet decoder. The MiT-B0 branch, however, produces feature maps with its own channel sizes and resolution pattern. This created a mismatch between the transformer branch outputs and the decoder inputs.

To solve this, we used 1x1 convolution projection layers to align the MiT-B0 feature channels with the feature dimensions required by the QuadPath decoder. A 1x1 convolution changes the number of channels in a feature map without significantly changing its spatial structure. This allowed the transformer features to be reshaped into a form that could be combined with the other expert branch features.

The model used UNet-style skip connections to pass intermediate encoder features directly to the decoder. In a UNet-style architecture, the encoder gradually compresses the image into deeper and more abstract features, while the decoder upsamples those features into a full-resolution segmentation mask. However, during compression, fine details such as boundaries and small structures can be lost. Skip connections reduce this problem by sending earlier, higher-resolution features directly to the matching decoder stage.

In our QuadPath model, skip connections were extended across all four expert branches. At each matching encoder depth, the features from the CNN, DCN, DSC, and MiT-B0 branches were concatenated channel-wise and provided to the corresponding decoder block. This allowed the decoder to use several types of information at the same scale: texture features from the CNN branch, deformation-aware features from the DCN branch, curved boundary features from the DSC branch, and global context features from the MiT-B0 branch.

The deepest features from the four branches were also combined at the bottleneck. The bottleneck can be understood as the most compressed representation of the image. At this stage, the model has reduced the spatial size of the feature maps but increased their semantic meaning. By concatenating the deepest features from all four branches, the bottleneck became a fused summary of the different expert perspectives before the decoder began reconstructing the segmentation mask.

5.5 Handling Channel and Resolution Mismatch

One of the major challenges that we faced was that the MiT-B0 feature maps did not naturally match the existing TriPath feature maps. The channel counts differed, and some feature maps were produced at different scales. Because the decoder expects specific dimensions, directly concatenating the transformer outputs with the convolutional branch outputs would cause shape errors.

To address this, the transformer outputs were projected into compatible channel dimensions before fusion. For deeper layers, the outputs could be combined more naturally because

their spatial sizes were closer to the expected decoder levels after downsampling. For shallower or higher-resolution layers, extra care was needed because the transformer branch does not preserve full-resolution details in the same way as the convolutional branches. This is why the implementation relied on aligning the transformer feature maps to the decoder stages where the spatial and channel dimensions could be made compatible with each other.

There were multiple ways this integration could have been implemented. A possible alternative would have been to use the MiT-B0 branch only at the bottleneck, where the deepest global features are fused with the TriPath bottleneck. This would be simpler because fewer skip-level feature maps would need to be aligned. However, this design would provide less transformer information to the decoder and fuse its final prediction with the TriPath output. This would make the transformer more independent, but it would reduce the direct multiscale interaction between the expert branches. Our approach used projected multiscale transformer features because it preserved the idea of expert branch diversity while still allowing the decoder to combine information across the four branches.

6 Evaluation and Results

6.1 Experimental Settings

Both the QuadEB and TriEB were evaluated on the same preprocessed DSAD dataset across four metrics. mIoU and mDice were used to understand the overall regional pixel-wise overlap between the predicted and ground-truth segmentation masks. MASD and NSD were used to evaluate key aspects of boundary prediction which emphasized how well the models performed on the difficult thinly curved anatomical structures present within laparoscopic imaging.

Input imaging was resized to 256x256 pixels from the original dataset resolution of 1920 x 1080 for the convolutional branches while the MiT-B0 branch had some additional resizing to 1024x1280 and were normalized using the ImageNet mean and standard deviation to match the pretraining distribution of the backbone. All models were optimized with Adam and trained for a fixed schedule of 80 epochs with a learning rate of 1×10^{-4} and a weight decay of 1×10^{-3} . These were run on a NVIDIA L40S GPU with a batch size of 4. Due to time constraints, output logs were parsed to extract the best value achieved for each metric across each organ.

To ensure a fair comparison, TriEB was retrained using the source code provided under identical experimental conditions to those used to evaluate QuadEB. This is due to the original paper reporting only the per organ mIoU and only the other metrics averaged across all organs. Due to this limitation this comparison will primarily utilize comparisons across the sample ran at the beginning of this experiment instead of the primary results reported in prior literature.

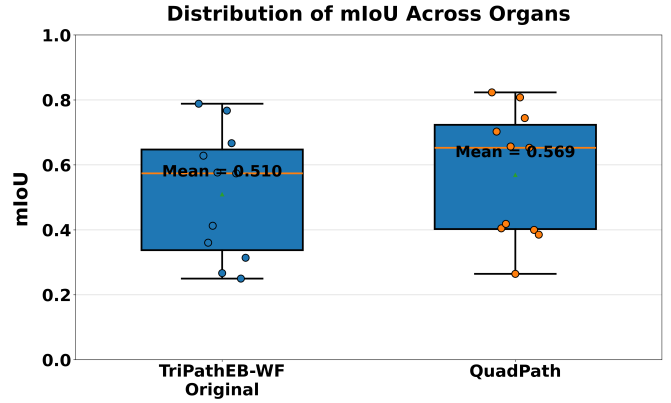


Figure 2: Boxplot distribution of the mIoU scores across the 11 DSAD organ classes for TriPathEB-WF and QuadPath. The results show a higher mean mIoU for QuadPath, with QuadPath achieving 0.5691 compared to 0.5095 for TriPathEB-WF.

6.2 Primary Results

QuadEB consistently outperformed TriEB across most evaluation metrics. Averaged across all 11 included DSAD organs, QuadEB achieved a mean mIoU of 0.5691 compared to 0.5095 for TriEB, as shown in Fig. 2. Similar improvements were also observed for mDice, where QuadEB achieved 0.6635 compared to 0.6053 for TriEB, and for NSD, where QuadEB achieved 0.5769 compared to 0.5079 for TriEB. In addition, QuadEB reduced the average MASD from 16.310 to 12.655. Since lower MASD indicates better boundary localization, this suggests that QuadEB produced more accurate organ boundaries across the evaluated organs.

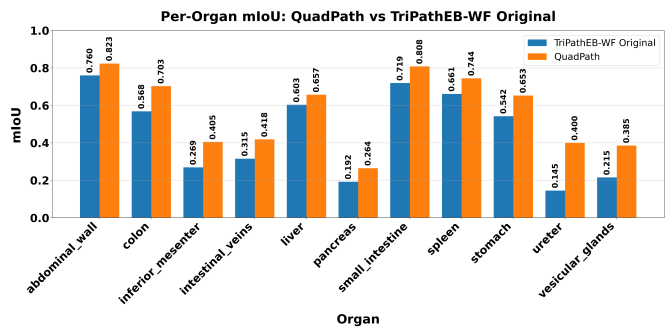


Figure 3: Per-organ mIoU comparison between TriPathEB-WF and QuadPath across the 11 DSAD organ classes. QuadPath improved mIoU for most organs, with notable gains in challenging classes such as the ureter, inferior mesenteric artery, liver, stomach, and spleen.

Looking at key organs where TriEB performed poorly, the ureter, inferior mesenteric artery, and vesicular glands showed clear improvement, as shown in Fig. 3. The ureter showed a substantial increase in mIoU, improving from 0.250 to 0.400, which is a 0.150 increase. Strong gains were also observed in the liver, stomach, and spleen, with improvements of approximately 0.080, 0.079, and 0.078, respectively. These organs

can exhibit challenges such as nonrigid deformation, boundary ambiguity, and large surface area variability. These are conditions where global contextual features were expected to provide the greatest benefits.

Overall, these results suggest that adding the MiT-B0 Vision Transformer branch improved the original expert-branch architecture by introducing broader contextual information. The improvements in mIoU and mDice indicate stronger regional overlap between the predicted and ground-truth masks, while the reduction in MASD and increase in NSD suggest improved boundary quality. This supports the hypothesis that transformer-based global context can complement the CNN, DCN, and DSC expert branches in laparoscopic organ segmentation.

6.3 Exploratory Findings

In order to fully understand the contributions of the ViT to the Quadpath an initial test was also conducted to compare the results across a solo run of the MiTB0 ViT used in QuadEb vs QuadEB's results. This comparison was conducted under the same experimental conditions as the primary evaluation. The scope of this analysis was constrained by limited GPU access and time availability. These results should therefore be interpreted as preliminary observations rather than strict conclusions.

Averaged for each metric the ViTSolo performed essentially the same with a extremely marginal difference of 0.0004 (average mIoU of 0.5695 vs 0.5691 for QuadEB). QuadEb did marginally better on mDice with an average of 0.6635 vs 0.6613 for ViT. ViT alone did marginally better on MASD (0.1773 reduction in average MASD) and marginally worse on NSD (0.0059 reduction in average NSD).

ViTSolo performed similarly to QuadPath across most metrics. In 6 of the 11 organs ViT marginally outperformed QuadPath in mIoU. These organs were primarily smaller globally dependent structures like the pancreas (0.2644 vs 0.2849 mIoU) and small intestine. The QuadEB model does better on organs with complex geometric patterns such as ureter (0.3997 vs. 0.3637 mIoU) and the liver (0.6568 vs. 0.6468 mIoU). Overall differences across mIoU are moderate with the largest absolute difference being 0.02 and the most minimal difference being 0.0018. Most drastically in these results is the difference in NSD (0.4933 vs 0.4600) and MASD 14.9958 vs 16.7021) for the ureter. This significant improvement for this organ suggests that the input from the cnn based experts helps to delineate thin tubular structures/ Comparison of singular organs suggest that the convolutional expert branches offer complementary information not fully captured by the transformer alone.

7 Discussion, Limitations, and Ethical Considerations

7.1 Summary discussion

The results suggest that adding a fourth expert improved the original TriPath architecture by giving the model access to broader contextual information. The QuadPath model showed higher average mIoU, mDice, and NSD while reducing MASD, meaning the predicted masks improved in both region overlap and boundary quality. This supports the idea that the MiT-B0 Vision Transformer branch added useful global context that complemented the CNN, DCN, and DSC branches.

7.2 Future Directions

The ViT preprocessing size could be changed to understand the effect on the output. The theorized effect currently is that larger input sizes may preserve more spatial detail for the transformer branch, which could improve small-organ segmentation and boundary precision. However, increasing input size would also increase memory usage and training time, so future experiments should compare multiple image sizes to determine whether the accuracy gain is worth the additional computational cost.

7.3 Limitations

Due to the nature of this study being constrained to a single semester as well as key difficulties accessing resources (no VPN, no code, and no access to a GPU cluster for almost 2 months). These models were implemented and tested on a rapid timeline. Additionally, there may be some issues within the dataset. Upon inspection, the segmentation masks appear to include the surgical devices within the images as a part of the segmentation mask, which changes the inherent geometric structure the model is trying to learn. While the paper behind the DSAD data offered no clear explanation for this, further testing across different laparoscopic images datasets may offer clearer insights into whether this truly created a problem within the model.

There is also a key consideration of how the ViT may have dominated the models overall encoding of features. While the vision transformer was added to serve as a complementary global-context expert, it is possible that its features contributed more strongly than the other branches during fusion. Because the model concatenates features from all four branches, the decoder may learn to rely more heavily on the transformer features if they are more informative or easier to optimize. This does make it difficult to know exactly how much each individual expert branch contributed to the final prediction. Future studies should test the model with different branch combinations, such as TriPath only, ViT only, TriPath and ViT at the bottleneck only, and the full QuadPath model. This would make it clearer whether the improvement came from

true expert-branch diversity or mainly from the added transformer backbone.

Another limitation is that this project focused on still-image segmentation rather than real-time surgical video. Although the long-term goal is to support live surgical guidance, the current model has not been evaluated for inference speed, temporal consistency, or deployment in the actual robotic surgical system. Therefore, the results should be interpreted as a step toward surgical scene understanding rather than a finished clinical tool.

7.4 Ethical Considerations

Because this project relates to surgical image analysis, ethical considerations are important even though the model is not being deployed clinically. A segmentation model used in surgery could influence how a surgeon interprets the surgical scene, so incorrect organ labeling could create risks if the output were trusted without human verification. For this reason, any future use of this type of system should keep the surgeon as the final decision-maker and treat AI outputs as assistive information rather than automatic truth.

There are also concerns related to dataset quality and generalization. If the model is trained on a limited dataset, it may not perform equally well across different patients, hospital, camera systems, lightning conditions, or surgical procedures. This could create unsafe or biased performance if the model works well in some settings but fails in others. Future work should test the model on larger and more diverse datasets before considering any clinical application.

8 Appendices

8.1 Appendix 1: Full Table of Results

Organ	Tri mIoU	Tri mDice	Tri MASD	Tri NSD	Quad mIoU	Quad mDice	Quad MASD	Quad NSD
Abdominal Wall	0.7886	0.8688	5.7115	0.7285	0.8232	0.8943	4.3284	0.7650
Colon	0.6282	0.7374	13.5144	0.5603	0.7026	0.7979	10.4963	0.6390
Inferior Mesenter	0.3604	0.4835	14.0131	0.3762	0.4048	0.5408	12.8809	0.4267
Intestinal Veins	0.4126	0.5183	15.5295	0.4974	0.4183	0.5275	14.8925	0.5106
Liver	0.5765	0.6690	16.7844	0.5231	0.6568	0.7374	12.0273	0.6059
Pancreas	0.2665	0.3597	32.7120	0.2870	0.2644	0.3509	28.0622	0.3033
Small Intestine	0.7672	0.8520	8.0613	0.7320	0.8078	0.8802	6.5622	0.7824
Spleen	0.6667	0.7441	11.4087	0.7037	0.7443	0.8121	5.3082	0.7803
Stomach	0.5739	0.6663	13.8403	0.5495	0.6526	0.7428	12.0547	0.6331
Ureter	0.2501	0.3351	26.1186	0.3170	0.3997	0.5128	14.9958	0.4933
Vesicular Glands	0.3142	0.4242	21.7211	0.3124	0.3854	0.5023	17.6602	0.4059
Average	0.5095	0.6053	16.3104	0.5079	0.5691	0.6635	12.6346	0.5769

Figure 4: Full comparison of TriEB vs QuadEb compared across each organ and each of the 4 metrics.

8.2 Appendix 2: Reproducibility

This project was conducted on the SLU Libra HPC cluster. Differences in other clusters may impact the accuracy of some commands provided.

To reproduce this project, first download the DSAD dataset into the workspace being used for training. In our experiments,

the dataset was stored outside of the GitHub repository because of its large file size. The repository should only contain the source code, model files, training scripts, and configuration files, while the dataset should be referenced through a local path on the cluster.

We used the following general data organization:

```
project_root/
  src/
  train_EB.py
  requirements.txt
  experiments/
  data/
    DSAD/
      images/
      masks/
```

The raw DSAD data should not be modified directly. Instead, preprocessing should create a separate processed version of the image-mask pairs. This makes it possible to rerun preprocessing without losing the original dataset.

A recommended setup is:

```
data/
  raw/
    DSAD/
  processed/
    DSAD/
      images/
      masks/
```

After the dataset is downloaded, create and activate a Python or Conda environment with the required dependencies. The main dependencies include PyTorch, torchvision, Hugging Face Transformers, NumPy, pandas, scikit-image, OpenCV, matplotlib, and any additional libraries required by the original Expert Branches implementation.

Example environment setup:

```
conda create -n expertbranches python=3.10
conda activate expertbranches
pip install -r requirements.txt
```

If using a GPU cluster, make sure the installed PyTorch version is compatible with the available CUDA version. On some HPC systems, PyTorch and CUDA compatibility may require installing PyTorch separately before installing the remaining requirements.

The DSAD preprocessing script should then be run to organize image-mask pairs, resize images, and prepare the masks for single-organ binary segmentation. In our project, the images were resized for the convolutional branches, while the MiT-B0 Vision Transformer branch required additional preprocessing and ImageNet normalization.

A general preprocessing command follows this structure:

```
python src/data/dsadb_preprocess.py \
  --data-root data/raw/DSAD \
  --output-root data/processed/DSAD
```

After preprocessing, training can be launched using the main training script. The exact file paths may need to be changed depending on the cluster directory structure.

Example training command:

```
python train_EB.py \
  --data-root data/processed/DSAD \
  --images-subdir images \
  --masks-subdir masks \
  --img-size 256 256 \
  --num-classes 2 \
  --epochs 80 \
  --batch-size 4 \
  --lr 1e-4 \
  --weight-decay 1e-3 \
  --save-dir experiments/quadpath_dsad \
  --run-name quadpath_mitb0
```

For GPU jobs on SLURM-based clusters, the training command should be placed inside an sbatch script. The script should request a GPU node, activate the correct environment, move into the project directory, and then run the training command.

Example SLURM structure:

```
#!/bin/bash
#SBATCH --job-name=quadpath_train
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=24:00:00
#SBATCH --output=logs/quadpath_%j.out
#SBATCH --error=logs/quadpath_%j.err
```

```
source ~/.bashrc
conda activate expertbranches
```

```
cd /path/to/project_root
```

```
python train_EB.py \
  --data-root data/processed/DSAD \
  --images-subdir images \
  --masks-subdir masks \
  --img-size 256 256 \
  --num-classes 2 \
  --epochs 80 \
  --batch-size 4 \
  --lr 1e-4 \
  --weight-decay 1e-3 \
  --save-dir experiments/quadpath_dsad \
  --run-name quadpath_mitb0
```

Training outputs should be saved inside the experiment directory. Important files include model checkpoints, epoch-level metrics, and step-level logs. In our experiments, output logs were parsed to extract the best values for each metric across each organ.

Expected output structure:

```
experiments/
  quadpath_dsad/
    latest.ckpt
    best.ckpt
    metrics_epoch.csv
    metrics_step.jsonl
```

For evaluation, the main metrics were mIoU, mDice, MASD, and NSD. Higher values are better for mIoU, mDice, and NSD, while lower values are better for MASD. The final tables and plots were generated by averaging the metric outputs across the 11 DSAD organ classes and comparing QuadEB against the retrained TriEB baseline.

To improve reproducibility, future users should record the following information for each run:

- Git commit hash of the code used.
- Dataset path and preprocessing settings.
- Python, PyTorch, CUDA, and Transformers versions.
- GPU type used for training.
- Random seed.
- Batch size, learning rate, weight decay, and number of epochs.
- Model checkpoint used for evaluation.
- Exact command or SLURM script used to launch training.

Because this project depends on GPU-specific libraries, results may vary slightly across environments. In particular, differences in CUDA versions, PyTorch versions, and GPU hardware may affect training speed, memory usage, and numerical reproducibility.

8.3 Appendix 3: Team Contribution Statement

This project was created with most parts being developed collaboratively in tandem. We worked together with Lin to strictly define the goals, scope and objectives of this project. Throughout the beginning of the project, Ash experienced some technical difficulties due a lack of access to Libra and the required VPN to work off campus. Due to this Ramez took on many of the initial stages of set up and exploration. Once both partners were able to access and work on Libra we collaboratively worked on restructuring the git hub repository to fit with the requirements of the assignment, planning which ViT to use, and getting our data set up for testing. For most steps throughout the process if one person developed it and ran it, another person created tests or analyzed the output. In the final skip connections we had 2 slightly different approaches we developed separately. Due to time constraints, the primary decision of which approach was determined by which ran without errors first was used as the primary decoder method. The remnants of Ash's bug filled fusion stage lie in a leftover branch

within the repository. Overall this process was highly iterative as each step often required us to go back and change or add something else within the code. For example, Ash set up the initial MiT encoder but Ramez had to change aspects in order for it to align with the skip connection architecture he implemented. The exploratory experiments and results on the singular ViT were primarily done by Ash so that Ramez could focus on the poster creation/presentation as well as updating the Git Hub Readme file and instructions. Overall contribution was equal across the different areas with each partner playing to their strengths and helping each other at every step.

9 References

- [1] L. Lingard, S. Espin, S. Whyte, G. Regehr, G. R. Baker, R. Reznick, J. Bohnen, B. Orser, D. Doran, and E. Grober, "Communication failures in the operating room: An observational classification of recurrent types and effects," *Quality and Safety in Health Care*, vol. 13, no. 5, pp. 330–334, Oct. 2004.
- [2] L. Guo, C. Camerota, M. Mahmoud, and F. Esposito, "Expert Branches: Module Diversity for Stronger Feature Learning in Laparoscopic Segmentation," *Proceedings of Machine Learning Research*, MIDL 2026, 2026.
- [3] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *International Conference on Learning Representations*, May 2021, [Online]. Available: <https://openreview.net/pdf?id=YicbFdNTTy>
- [4] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 12077–12090.
- [5] M. Carstens et al., "The Dresden Surgical Anatomy Dataset for Abdominal Organ Segmentation in Surgical Data Science," *Scientific Data*, vol. 10, no. 1, p. 3, Jan. 2023, doi: 10.1038/s41597-022-01719-2.